

Automated Railway Interlocking & Collision Avoidance Engine

A software engineering simulation demonstrating concurrent resource coordination, mutual exclusion, and safety-oriented interlocking principles.

The Problem: Shared Tracks, Concurrent Requests

Route Conflict

Two trains request overlapping routes that share one or more tracks simultaneously.

Occupancy Conflict

A train attempts to enter a track that is already occupied or reserved by another thread.

Deadlock Risk

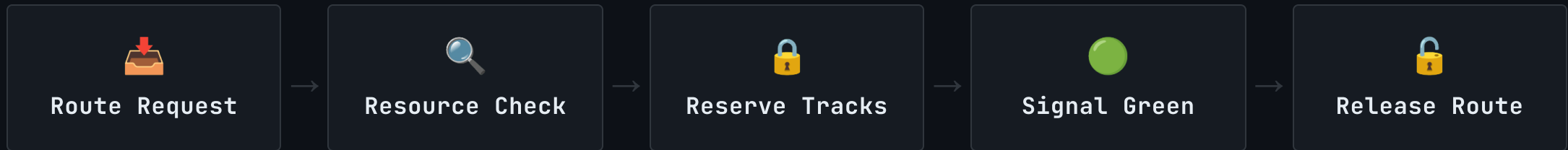
Circular waiting where multiple trains wait indefinitely for resources held by each other.



Conceptual Conflict: Concurrent access to shared track resource

Simple "move train" logic fails in concurrent environments.
Software must coordinate shared resources safely.

Interlocking: Check, Reserve, Authorize



Railway interlocking is a safety concept where conflicting movements are prevented by controlling access to shared resources. In this project, we implement this as a software abstraction: [InterlockingEngine](#).

Core Architecture: Engine First, UI Second

Streamlit Dashboard

Python-based system monitoring

Flask Web UI

SVG-based network visualization

↑ (Shared Engine Logic) ↑

Models

- Station / Track
- Signal / Route
- Train / Event
- Network

Engine Layer

- Interlocking Engine
- Train Simulator
- Collision Monitor
- Event Processor

Utilities

- Thread-Safe Queue
- System Logger
- Signal Controller
- System Monitor

A Simplified Railway Network Model

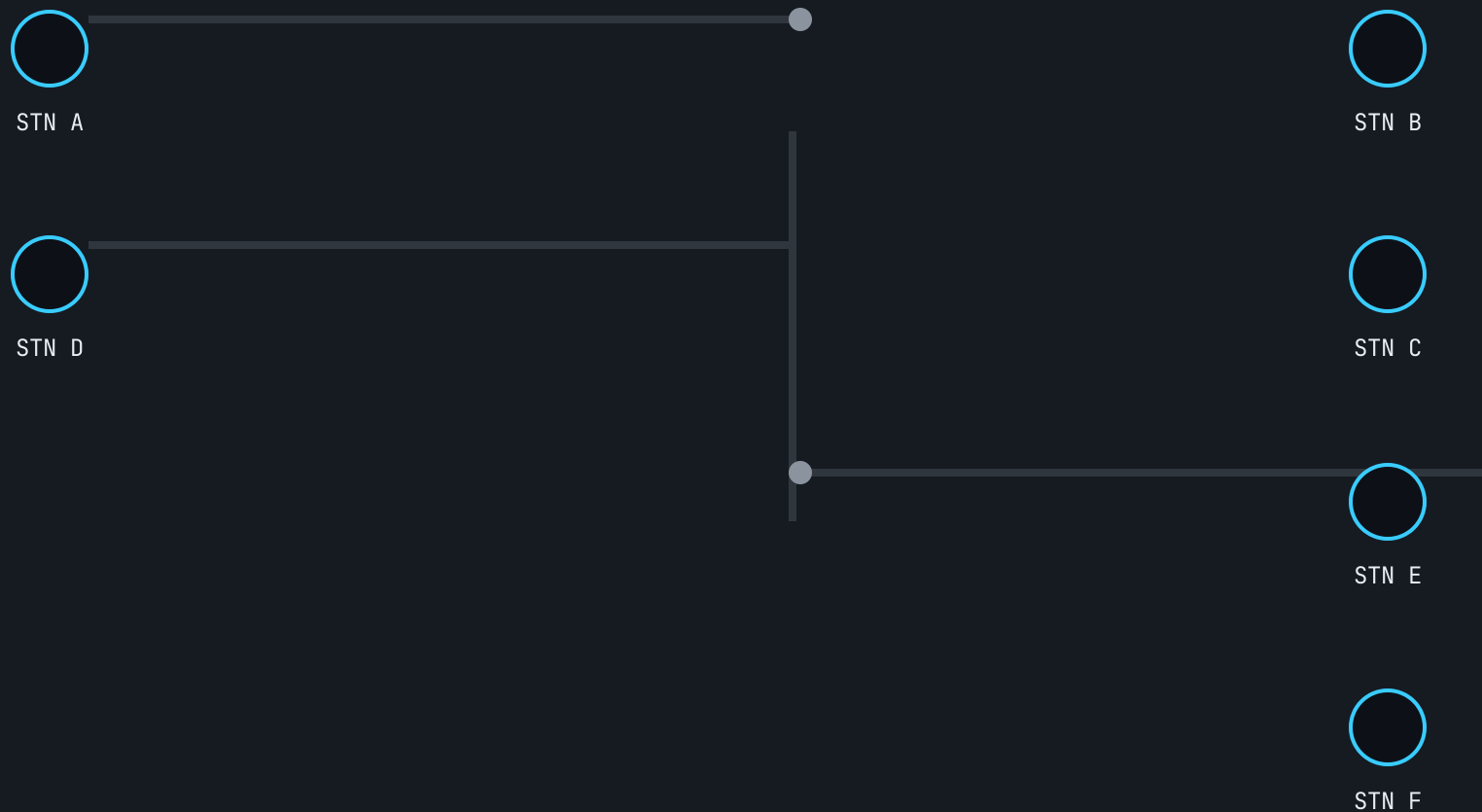
Stations	06
----------	----

Tracks	10
--------	----

Signals	10
---------	----

Routes	10
--------	----

The engine uses an Object-Oriented model where each entity maintains its own state, locks, and relationships.



Route Reservation + Mutual Exclusion

Whole-Route Reservation

Before a train moves, the engine checks and reserves **every track** in the requested route. This prevents mid-route conflicts.

Mutual Exclusion (Mutex)

Each track object contains a `threading.Lock`. Only one train thread can modify a track's state at a time.

Atomic Checks

The Interlocking Engine uses a global lock during the "Check & Reserve" phase to ensure no two trains are granted conflicting routes.

models/track.py

```
class Track:
    def __init__(self, track_id):
        self.id = track_id
        self.lock = threading.Lock()
        self.reserved_by = None
        self.occupied_by = None
    def reserve(self, train_id):
        with self.lock:
            if self.reserved_by is None:
                self.reserved_by = train_id
                return True
            return False
```

🔒 The `with self.lock` block ensures thread-safe access to the track's reservation state.

Concurrent Trains, Thread-Safe Events

Multithreading Model

Train Thread T1 RUNNING

Train Thread T2 RUNNING

Train Thread T3 WAITING

Train Thread T4 RUNNING

Each train operates in its own thread, simulating independent movement. Concurrency is managed via shared resource locks.

Event Flow

Train Threads emit events



Thread-Safe queue.Queue

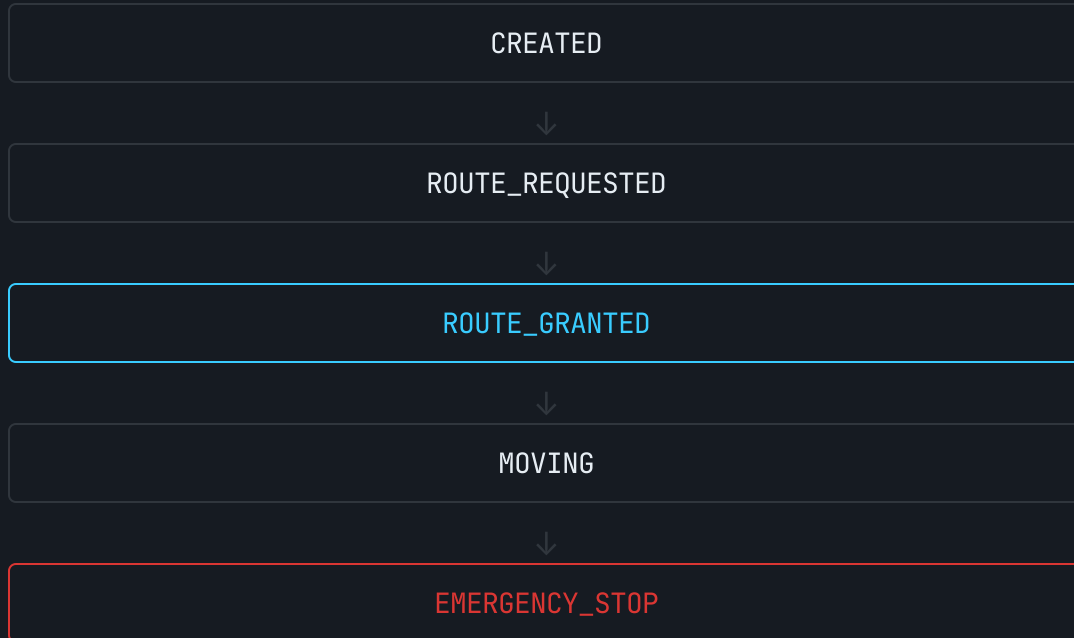


Event Processor

Decouples simulation logic from monitoring and UI updates.

States, Signals, and Defensive Monitoring

Train State Machine



Safety Mechanisms



GREEN: Route Authorized



RED: Track Blocked / Default

CollisionMonitor

A defensive background thread that checks for overlapping track occupancy. If a conflict is detected, it triggers COLLISION_WARNING and EMERGENCY_STOP.

Controlled Scenarios Make the Logic Visible

1. Route Conflict

Two trains request routes sharing a track. Interlocking grants one and makes the other wait.

RESULT: Safe resource contention management.

2. Track Occupancy

A train tries to enter a track already in use. The engine rejects the request until the track is released.

RESULT: Prevention of overlapping occupancy.

3. Collision Prevention

Simulated defensive check: if two trains are detected on the same track, an emergency stop is triggered.

RESULT: Fail-safe defensive monitoring.

4. Concurrent Requests

Multiple trains request resources simultaneously. Thread-safe locks ensure atomic state updates.

RESULT: Robust multithreaded coordination.

Observability: Logs, Monitor, and Dashboards

Streamlit Dashboard

Real-time system status monitoring using Python's Streamlit framework. Displays train states, track occupancy, and active routes in a clean tabular format.

Flask Web UI

Interactive SVG visualization of the railway network. Shows train movement and signal changes dynamically, making the simulation logic accessible.

System Logging

```
[2026-09-10 22:00:01] INFO: Train T1 started
[2026-09-10 22:00:05] INFO: Route R1 granted to T1
[2026-09-10 22:00:10] INFO: T1 entered track TR1
[2026-09-10 22:00:15] WARN: T2 waiting for TR1
[2026-09-10 22:00:20] INFO: T1 left track TR1
[2026-09-10 22:00:21] INFO: TR1 released
```

Traceability for debugging and performance analysis. Every state change and resource event is recorded in `logs/railway.log`.

Evidence and Honest Boundaries

Automated Test Suite

12 / 12 PASSED

test_interlocking.py	3 tests	PASSED
test_collision.py	2 tests	PASSED
test_concurrency.py	1 test	PASSED
test_deadlock.py	1 test	PASSED
test_signal_controller.py	2 tests	PASSED
test_stress.py	1 test	PASSED

No deadlocks were observed in the tested concurrent scenarios.

Project Limitations

Educational Scope

Designed for software engineering demonstration, not for real-world railway safety-critical operation.

No Formal Proof

The system uses robust synchronization but does not constitute a formal mathematical proof of deadlock freedom.

Software Simulation

Collision detection and emergency stops are simulated defensive mechanisms within the software environment.

From Railway Problem to Software Insight

The project demonstrates how software engineering concepts—**Object-Oriented Modeling, Concurrency, Mutual Exclusion, and Event-Driven Processing**—can be combined to simulate the safe coordination of shared resources.

It provides a tangible bridge between abstract computer science principles and a complex real-world safety problem.

[Python](#)[Threading](#)[Locks](#)[Queues](#)[Flask](#)[Streamlit](#)

View Project on GitHub: [/railway-interlocking-engine](#)